SCA Backend: CRUD, Encryption & Firestore Guide

How each create / read / update / delete request is processed, what is encrypted, and where every record is stored.

Project: **sca-be** (Node 20, TypeScript, Express 5, Zod) · Database: **Firebase Firestore** · File storage: **Cloudflare R2** · Updated: 30 October 2026

1. How a request is processed
2. How encryption works
3. Firestore collections map
4. CRUD procedures by module
5. Cloudflare R2 storage
6. Encrypted data summary
7. Fields that stay plain
8. Rate limits

ENCRYPTED AES-256-GCM, can be decrypted by the server

HASHED one-way, cannot be decrypted

PLAIN stored as normal text

1. How a request is processed

Every request goes through the same pipeline before it touches the database. Code lives in a feature folder: `src/modules/<feature>/<feature>.{routes, controller, service, validator, types}.ts`.

1. **Security layer** (`app.ts`)
   Helmet security headers, CORS that only accepts the frontend (`CLIENT_URL`), and a JSON body limit of 100 KB.

2. **Rate limit per IP** (`rateLimit.middleware.ts`)
   Max 300 requests per minute per IP address. Over the limit → `429`.

3. **Authentication** (`auth.middleware.ts → requireAuth`)
   Reads `Authorization: Bearer <JWT>`, verifies it, then checks the session is still active: not signed out, and its version matches the user's (sign out of all devices and password resets raise it). Sets `req.user`. Otherwise → `401`. Only `/auth/*` routes are public.

4. **Rate limit per user**
   400 requests per 10 minutes per signed-in user (plus stricter limits on login, forgot password and uploads; see section 8).

5. **Role check** (`role.middleware.ts → requireRole("dev")`, developer routes only)
   Reads the user's `accessRoles` from Firestore on every request, so role changes work right away. No matching role → `403`.

6. **Validation** (`validate.middleware.ts` + `*.validator.ts`)
   Zod checks and cleans the body, query and URL params (trims text, enforces lengths, formats, required fields). Invalid → `400` with field errors. Files are checked by type, size (images 10 MB, documents 50 MB) and **file signature** (the content must really be that type).

7. **Controller** (`*.controller.ts`)
   Takes the user id and the validated input and calls the service. No business logic here.

8. **Service** (`*.service.ts`)
   Business logic. Builds the path `users/{userId}/{collection}` so a user can only reach their own records (another user's id → `404`). **Encrypts sensitive fields before writing** and **decrypts them after reading**.

9. **Storage**
   Records go to Firestore; file bytes are encrypted and go to Cloudflare R2.

10. **Activity log** (`modules/audit`)
    Writes `{ action, entity, entityId, createdAt }` to `users/{userId}/auditLogs` for sign-ins, sign-outs, password resets, role changes and every create/update/delete/download of students, cases, appointments and files. Never the record's content. 10 deletions or 20 downloads in 10 minutes email the user an alert.

11. **Response**
    Success: `{ success: true, data, pagination? }`. Error: `{ success: false, message, details? }`. Create → `201`, delete → `204`. Raw server errors are never sent to the browser.

Lists: every list is paginated (max 15 per page) and newest first. Filters Firestore can do run in the query; text search and filters on encrypted fields run on the server after decrypting, then the result is paged.

2. How encryption works

Method	Algorithm	How it is used
ENCRYPT text / JSON	AES-256-GCM (`utils/encryption.ts`)	Sensitive fields are encrypted before saving and decrypted only for the signed-in owner. Stored as one string: `iv:authTag:ciphertext` (base64). A new random IV is used every time, so the same value never looks the same twice. Groups of fields are encrypted together as JSON (`encryptJson`).
ENCRYPT files	AES-256-GCM (binary)	File bytes are encrypted before uploading to R2: `[12-byte IV][16-byte authTag][ciphertext]` . Downloaded files are decrypted on the server and streamed to the browser.
HASH passwords	bcrypt, 12 rounds	Passwords are never stored or decrypted; login compares the typed password with the hash.
HASH email-link tokens	SHA-256	Only the hash is saved as the `authTokens` document id. The raw token exists only inside the email link.
HASH lookups	HMAC-SHA256 with the encryption key	`emailHash` (sign-in and duplicate checks) and `phoneHash` (duplicate mobile numbers) find accounts without decrypting. Rate-limit counters are also keyed by a hash, so no IP address is stored.
HASH sign-in codes	SHA-256	The 6-digit code of the two-step sign-in is stored only as a hash (10 minutes, 5 tries).
Key rotation	—	To replace the key: set `ENCRYPTION_KEY_PREVIOUS` (old) and `ENCRYPTION_KEY` (new), deploy, run `npm run db:rotate-key` . Reading tries both keys until everything is re-encrypted.

Encryption key: `ENCRYPTION_KEY` (64 hex characters) in `.env` / Render environment. If it is lost or changed without the rotation steps, every encrypted field and uploaded file becomes unreadable. Back it up offline and never commit it.

3. Firestore collections map

Personal records live **under their owner**, so ownership is enforced by the path itself. Shared data (support reports, system updates, tokens) is top-level.

```
Firestore
├── users/{userId}                      account + profile → name, email, job title,
│                                       school, phone, address ENCRYPTED
│   ├── students/{id}                   student records          → data ENCRYPTED
│   ├── cases/{id}                      case records             → data ENCRYPTED
│   ├── appointments/{id}               appointments             → data ENCRYPTED
│   ├── files/{id}                      uploaded file info       → data ENCRYPTED
│   │                                   (file bytes ENCRYPTED in R2)
│   ├── categories/{id}                 Settings dropdown lists  → data ENCRYPTED
│   ├── todo/board                      To Do board              → data ENCRYPTED
│   └── auditLogs/{id}                  who created/updated/deleted/downloaded what
│
├── supportRequests/{id}                bug reports + feature requests (incl. votes)
│                                       → data ENCRYPTED (+ media in R2)
├── updates/{id}                        published system updates → data ENCRYPTED
├── authTokens/{sha256(token)}          email links + sign-in codes (id HASHED,
│                                       invited email ENCRYPTED, code HASHED)
└── rateLimits/{hash}                   rate-limit counters (no IP stored)
```

users/{userId} fields

Field	Protection	Notes
`name`, `email`	ENCRYPTED	Email unique per account.
`emailHash`	HASHED	HMAC-SHA256: sign-in finds the account by this.
`password`	HASHED	bcrypt. `null` for Google-only accounts.
`phone`	ENCRYPTED	Normalized to `09XXXXXXXXX` before encrypting.
`phoneHash`	HASHED	HMAC-SHA256, used to block duplicate mobile numbers.
`address`	ENCRYPTED	JSON: region, province, city, barangay (PSGC `{ code, name }`) and street.
`accessRoles`	PLAIN	e.g. `["sca-1"]`, `["sca-1","dev"]`. Older accounts may have a single `accessRole`.
`role`, `school`	ENCRYPTED	Job title, and all school details as one encrypted JSON.
`tokenVersion`, `revokedSessions`	PLAIN	Session control: signed-out sessions and "sign out of all devices".
`photo`	PLAIN info	`{ storageKey, mimeType, updatedAt }`. The picture itself is encrypted in R2.
`firebaseUid`, `emailVerifiedAt`, `readUpdateIds`, `readNotificationIds`, `createdAt`	PLAIN	Google link, confirmation time, read notifications (newest 300 kept).

4. CRUD procedures by module

Every record a user owns (students, cases, appointments, files, categories, To Do board) is stored as `{ referenceNumber, data, createdAt, updatedAt }`. `data` is the whole record encrypted as one JSON string. Only the reference number and times stay plain (for sorting). Search and filters run on the server after decrypting.

4.1 Accounts & authentication → `users`, `authTokens`

CRUD	Endpoint	Procedure
Create	`POST /auth/register`	Validate name, email, password (8–72, letters + numbers, not a common password), PH mobile, address. **Email already registered** → email its owner ("you already have an account", or the confirmation again). Otherwise, in a **transaction**: reject a duplicate email hash or phone hash → hash password (bcrypt), encrypt name, email, job title, school, phone and address → create `users/{id}` (role `sca-1`) with encrypted default categories → email a confirmation link. **Mobile already used** → email the person. The response is always the same `{ email }`.
Read	`POST /auth/login`	Step 1: find the account by email hash, compare bcrypt hash, require a confirmed email → save a hashed 6-digit code in `authTokens` (10 minutes) and email it → return `{ challenge }`.
Create	`POST /auth/login/verify`	Step 2: check the code (5 tries, then the challenge is deleted) → create a session token (session id + version) → log "sign-in" → return token + decrypted profile.
Create	`POST /auth/google`	Verify the Firebase ID token → find the user by Firebase UID or email hash (link it), or create a new account → session token.
Update	`POST /auth/verify-email`, `/reset-password`	Hash the token from the link, find it in `authTokens`, check type and expiry, delete it (one use) → set `emailVerifiedAt`, or a new password hash and **sign out every device**.
Update	`POST /auth/logout`, `/auth/logout-all`	Add this session id to `revokedSessions`, or raise `tokenVersion` so every older token stops working → log it.
Create	`POST /auth/register-developer`	From an invitation link for a new email → verified account with role `dev`.
Update	`POST /auth/accept-invitation`	From an invitation link for an existing account → add `dev` to its roles only when the owner accepts → log "role-added".

4.2 Profile → `users/{userId}` + R2

CRUD	Endpoint	Procedure
Read	`GET /profile`	Read the user document → decrypt name, email, job title, school, phone and address → return `{ user, school }` (never the password hash).
Update	`PUT /profile`	Validate → reject if another account has the same email hash or phone hash (`409`) → encrypt every detail → update the document.
Create / Update	`POST /profile/photo`	Check type (PNG/JPG/WEBP), file signature and size (10 MB) → encrypt bytes → upload to R2 key `{userId}/profile-photo` (replaces the old one) → save photo info on the user.
Read	`GET /profile/photo`	Download from R2 → decrypt → send the image.
Delete	`DELETE /profile/photo`	Delete the R2 object → set `photo: null` .
Read	`GET /profile/activity`	The user's own activity log from `users/{id}/auditLogs` , newest first (Profile → Recent activity).

4.3 Students → `users/{userId}/students`

CRUD	Endpoint	Procedure
Create	`POST /students`	Validate (student name and date encoded required) → **encrypt the whole form as** `data` → add with a generated `recordId` → audit log.
Read	`GET /students` , `/students/:id`	Load newest first → decrypt → name search, date range and dropdown filters (sex, grade & section, …) run after decrypting → paginate.
Update	`PUT /students/:id`	Check it exists under this user (else `404`) → re-encrypt → **replace** the document → audit log.
Delete	`DELETE /students/:id`	Check it exists → delete → audit log.

PLAIN recordId, createdAt, updatedAt only.
ENCRYPTED `data` : the whole student form (name, date encoded, sex, grade & section, school year, birthday, LRN, address, contacts, guardians, interests, help requested, parent concerns, review, remarks, …).

4.4 Cases → `users/{userId}/cases`

CRUD	Endpoint	Procedure
Create	`POST /cases`	Validate (intake date, learner name, follow-up date required) → **encrypt the whole case as** `data` → add with a generated `caseNumber` → audit log.
Read	`GET /cases` , `/cases/:id`	Load newest first → decrypt → search by learner name, intake date range and status (overdue / today / scheduled / accomplished) after decrypting → paginate.
Update	`PUT /cases/:id`	Check it exists → re-encrypt → **replace** the document (older plain-text cases lose their plain fields) → audit log.
Delete	`DELETE /cases/:id`	Check it exists → delete → audit log.

PLAIN caseNumber, createdAt, updatedAt only.
ENCRYPTED `data` : learner name, grade & section, concern, action, referred to, classification, intake date, follow-up date, status, remarks.

4.5 Appointments → `users/{userId}/appointments`

CRUD	Endpoint	Procedure
Create	`POST /appointments`	Validate (student, date, time required) → **encrypt the whole appointment as** `data` → add with a generated `appointmentId` → audit log.
Read	`GET /appointments`, `/:id`	Load newest first → decrypt → search, dates, status and staff run after decrypting → paginate.
Update	`PUT /appointments/:id`	Check it exists → re-encrypt → **replace** the document → audit log.
Delete	`DELETE /appointments/:id`	Check it exists → delete → audit log.

PLAIN appointmentId, createdAt, updatedAt only. ENCRYPTED `data`: student, staff, date, time, duration, type, reason, status, reminder, rescheduled from, notes.

4.6 Files → `users/{userId}/files` + R2

CRUD	Endpoint	Procedure
Create	`POST /files`	Check type (pdf, xls, xlsx, csv, ppt, pptx, png, jpg), file signature and size (documents 50 MB, images 10 MB) → encrypt bytes → upload to R2 under a **random key** `{userId}/{uuid}` (the file name is never in the path) → save name, size, type and key **encrypted as** `data` → audit log.
Read	`GET /files`, `/files/:id/download`	List info (search by name, filter by kind). Download: read from R2 → decrypt → send → audit log (`download`).
Delete	`DELETE /files/:id`	Delete the R2 object → delete the record → audit log.

PLAIN createdAt only. ENCRYPTED `data` (name, type, size, storage key) and the file content (in R2).

4.7 Settings categories → `users/{userId}/categories`

CRUD	Endpoint	Procedure
Create	`POST /categories`	Decrypt the user's categories and reject a duplicate name (`409`) → add `{ name, options: [] }` **encrypted as** `data` → clear the cache.
Read	`GET /categories`	Served from a short in-memory cache per user, otherwise read newest first and paginate.
Update	`PATCH /categories/:id`, `/:id/options`	Decrypt → rename (no duplicates) or replace the options list → re-encrypt and save → clear the cache.
Delete	`DELETE /categories/:id`	Delete → clear the cache.

PLAIN createdAt only. ENCRYPTED `data`: name and options.

4.8 To Do board → `users/{userId}/todo/board`

CRUD	Endpoint	Procedure
Read	`GET /todo/board`	Read the single board document and decrypt it; new users get To Do / In Progress / Done.
Create / Update / Delete	`PUT /todo/board`	The whole board is saved at once (columns and cards) **encrypted as** `data`. Every card's column must exist; max 30 columns and 500 cards.

ENCRYPTED `data`: all columns and cards.

4.9 Support reports & feature requests → `supportRequests` (top-level) + R2

CRUD	Endpoint	Procedure
Create	`POST /support`	Any user. Validate type (bug / feature), subject, description, severity → encrypt up to 6 images/videos and upload to R2 under `support/{reportId}/{uuid}` → save the report: subject, description, severity, reporter name and email and attachment details **encrypted as** `data`. Feature requests start as *under review*.
Read	`GET /support/board`, `/support/:id`, `/features`	Decrypt → developers see the Kanban board; everyone sees feature requests (reporter hidden, only `mine`), searched after decrypting. Media is decrypted on download.
Update	`PATCH /support/:id`	Developers only: status (todo / in-progress / done), assignee (must be on the team), review (under review / approved / rejected).
Update	`POST /features/:id/vote`	In a **transaction**: add or remove the user in `voterIds` and update `votes` (one vote per user, approved requests only).

PLAIN type, status, review, reporterId, assigneeId, voterIds, votes, dates. ENCRYPTED `data`: subject, description, severity, reporter name/email, attachment details; media files in R2.

4.10 System updates → `updates` (top-level) + R2

CRUD	Endpoint	Procedure
Create	`POST /updates`	Developers only, for a done report. Sanitize the editor HTML (removes scripts and unsafe tags) → encrypt media and upload under `updates/{updateId}/{uuid}` → save title, text, media details and author name **encrypted as** `data` → notify every other user in real time.
Read	`GET /updates`, `/updates/:id`	Everyone. Paginated, newest first, decrypted. Media decrypted on download.
Update	`POST /updates/:id/read`, `POST /notifications/read`	Saves read ids on the user (`readUpdateIds`, `readNotificationIds`).

PLAIN reportId, reportType, authorId, publishedAt. ENCRYPTED `data`: title, text (sanitized HTML), media details, author name; media files in R2.

4.11 Team → `users` (roles)

CRUD	Endpoint	Procedure
Read	`GET /team`	Developers only. Users with the `dev` role (new and older role fields), each listed once, names and emails decrypted.
Create / Update	`POST /team/invitations`	Save a hashed invitation token in `authTokens` (72 hours, invited email encrypted) and email the link. New email → "create your developer account"; existing account → "accept invitation" (the role is added only if the owner accepts).

4.12 Reports & notifications (read only)

`GET /reports/dashboard`, `/reports/summary`, `/reports/notifications` read `cases` and `appointments`, decrypt them on the server and return **counts only**. Remarks and notes are never included. `GET /notifications/stream` is a real-time connection (Server-Sent Events) and stores nothing.

5. Cloudflare R2 storage

Every file is encrypted on the server before upload; R2 never receives readable content. Only the key is stored in Firestore, and keys are never sent to the browser.

R2 key	Linked from	Content
`{userId}/{uuid}`	`users/{id}/files/{id}.storageKey`	Uploaded documents and images **ENCRYPTED**
`{userId}/profile-photo`	`users/{id}.photo.storageKey`	Profile picture **ENCRYPTED**
`support/{reportId}/{uuid}`	`supportRequests/{id}.attachments[]`	Bug report images/videos **ENCRYPTED**
`updates/{updateId}/{uuid}`	`updates/{id}.media[]`	System update images/videos **ENCRYPTED**

6. Encrypted data summary

Where	Field	Protection	Reason
`users/{id}`	password	**bcrypt**	Login credential
`users/{id}`	name, email, job title, school	**AES-256-GCM**	Personal and school details
`users/{id}`	emailHash	**HMAC-SHA256**	Sign-in lookup without decrypting
`users/{id}`	phone	**AES-256-GCM**	Personal contact info
`users/{id}`	phoneHash	**HMAC-SHA256**	Duplicate check without decrypting
`users/{id}`	address (region, province, city, barangay, street)	**AES-256-GCM**	Home address
`users/{id}/students/{id}`	data (the whole student record)	**AES-256-GCM**	Student personal record
`users/{id}/cases/{id}`	data (the whole case)	**AES-256-GCM**	Confidential case record
`users/{id}/appointments/{id}`	data (the whole appointment)	**AES-256-GCM**	Confidential appointment
`users/{id}/files/{id}`	data (name, type, size, storage key)	**AES-256-GCM**	File names can include student names
`users/{id}/categories/{id}`	data (name, options)	**AES-256-GCM**	Dropdown values (staff names, sections)
`users/{id}/todo/board`	data (columns, cards)	**AES-256-GCM**	Cards can mention students
`supportRequests/{id}`	data (subject, description, severity, reporter, attachments)	**AES-256-GCM**	Reports can describe students
`updates/{id}`	data (title, text, media, author)	**AES-256-GCM**	Encrypt by default
`authTokens/{id}`	document id; invited email; sign-in code	**SHA-256** **AES-256-GCM**	Email links, invitations, two-step sign-in
`rateLimits/{id}`	document id	**HMAC-SHA256**	Counter per IP/user without storing the IP

Cloudflare R2	all files, photos and media	**AES-256-GCM**	May contain student records

Records saved before full encryption (including users, support reports and updates) are still readable and are converted once with `npm run db:encrypt-all` (add `-- --dry-run` to test first). Deploy the backend before running it.

7. Fields that stay plain

By default everything is encrypted (see CLAUDE.md). No personal or record content is plain; only these system fields, which the app needs for lookups, sorting and access checks:

Fields	Why
Document ids, `reporterId`, `assigneeId`, `authorId`, `reportId`, `updateId`, `firebaseUid`, case/record/appointment numbers	Link records together and find them
`accessRoles`, `tokenVersion`, `revokedSessions`, report `type` / `status` / `review`, `voterIds` / `votes`	Access checks, the Kanban board and voting
`createdAt`, `updatedAt`, `publishedAt`, `emailVerifiedAt`, `expiresAt`	Sorting newest first, expiry
`users/{id}/auditLogs`	Only the action, record type, record id and time (no content)
`readUpdateIds`, `readNotificationIds`, `photo.mimeType` / `storageKey`	Ids only; the photo itself is encrypted in R2

8. Rate limits

Over a limit, the API returns `429` with a "please wait" message and a `Retry-After` header. The first two limits are counted in memory; the others are stored in Firestore (`rateLimits`) so restarts don't reset them.

Applies to	Limit	Counted per
Every API request	300 per minute	IP address
Any request after sign-in (create, read, update, delete)	400 per 10 minutes	user
Sign-in (password or Google)	3 failed attempts per 10 minutes	IP address
Sign-in codes (step 2)	10 wrong codes per 10 minutes (and 5 per code)	IP address
Forgot password, new sign-in code	2 per 3 minutes (first + resend)	IP address
Register, resend confirmation	5 per hour	IP address
Email links (verify, reset, developer invite)	20 per 15 minutes	IP address
File uploads (Files page and profile photo)	6 per 10 minutes	user
Contact Support / feature requests	5 per 10 minutes	user
Developer invitations	20 per hour	user